Sergio Alfredo Junca Valero
ST1611 Computación en Nube · Universidad EAFIT
4 de noviembre de 2026

DIGITAL CAFÉ LUNA ARCHITECTURE DECISION BRIEF

Vender el pico de la campaña sin perder pedidos y sin darle al equipo más operación de la que puede sostener. Lo estático sale del borde, la API escala en dos AZ con un techo, los pedidos viven en una base Multi-AZ, todo se opera como código y lo intermitente corre por evento. Cuesta USD 173 en un mes normal y 292 con campaña.

1 PROBLEMA Y RESTRICCIONES

Digital Café Luna prepara una campaña nacional de pocos días con demanda incierta. Hoy compra capacidad antes de conocer la demanda: el pico la satura y el resto del año sobra. Necesita vender en el pico sin perder pedidos ni exponer datos de clientes, con un equipo pequeño y un presupuesto limitado.

Prioridad. Cuando chocan, R1 > R2 > R3 > R5: perder un pedido confirmado es peor que una página lenta, y el pico de la campaña es más probable que la caída de una AZ. R4 es un límite: descarto lo que el equipo no pueda operar.

Alcance. Web y campaña, pedidos, clientes y reportes. El inventario de las tiendas va en una segunda etapa: en la priorización del caso tiene valor medio y riesgo alto.

RESTRICCIÓN DEL CASO	QUÉ EXIGE A LA ARQUITECTURA	DECISIÓN
R1 Datos de clientes y pedidos	Ningún pedido confirmado se pierde; datos privados, cifrados y auditables.	D2, D3
R2 Demanda irregular por la campaña	Capacidad que sigue a la demanda, con techo de costo.	D1, D5
R3 Resiliencia	Seguir vendiendo si cae una instancia o una zona de disponibilidad (AZ).	D2
R4 Equipo tecnológico pequeño	Pocas piezas que operar: servicios administrados, todo como código y alarmas que actúan.	D2, D4, D5
R5 Costos controlados	Costo fijo mínimo y justificado; el variable, acotado.	D1, D3
R6 Capacidad de evolución	Sumar capacidades, como el inventario, sin rehacer lo que funciona.	D5

2 ARQUITECTURA

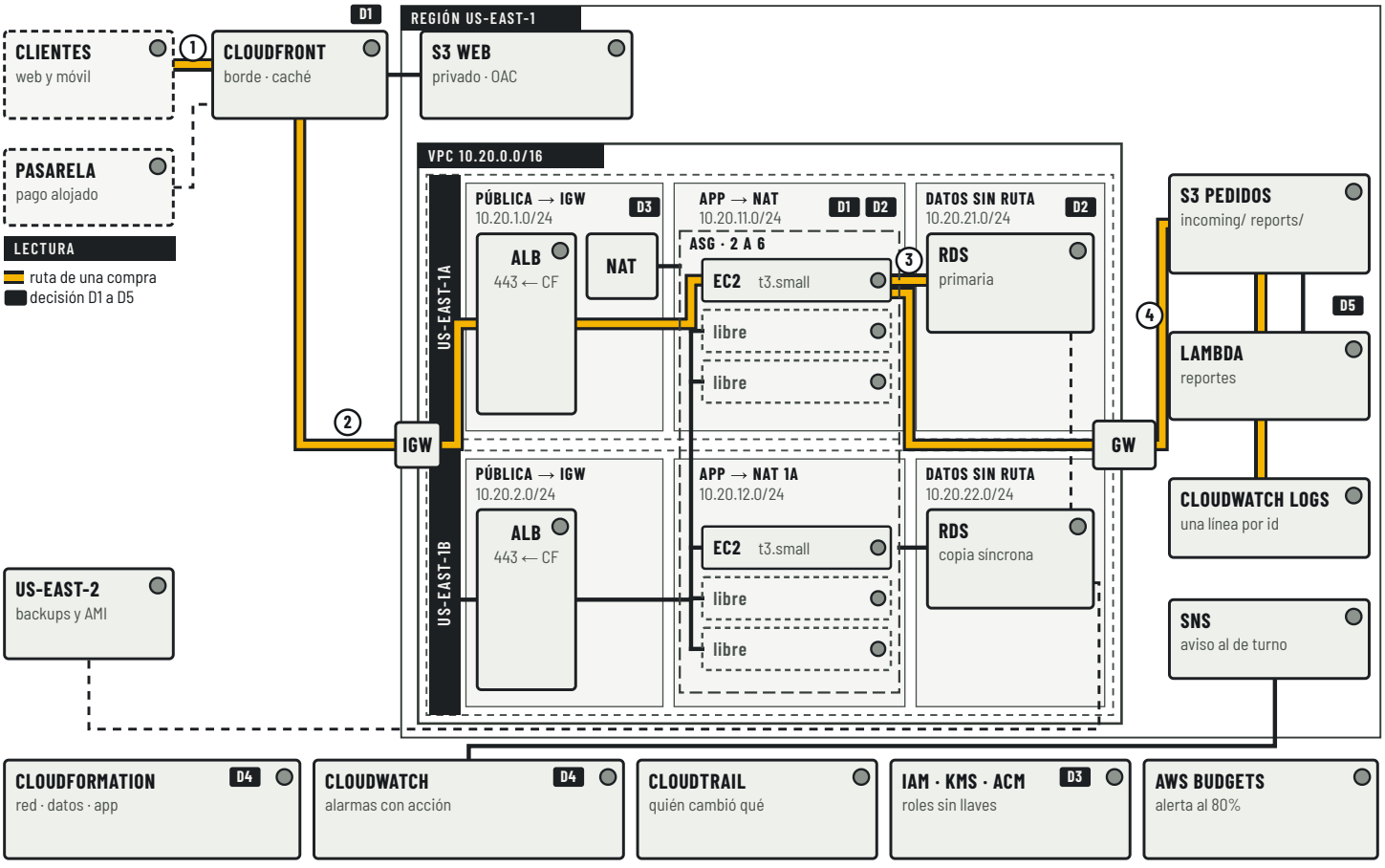


Figura 1. Arquitectura en us-east-1: la VPC 10.20.0.0/16 del Lab 01 con tres capas en dos AZ, y cada subnet con su CIDR y su ruta. Los números siguen una compra; D1 a D5 marcan dónde actúa cada decisión. La línea ámbar es la ruta de una compra.

(1) CloudFront entrega la web desde caché; solo él lee el bucket (OAC).
(2) La llamada a /api pasa al ALB, que solo acepta a CloudFront, y a una EC2 sana.
(3) La app guarda el pedido en RDS; la pasarela confirma el pago con un webhook firmado.
(4) El pedido confirmado deja incoming/<id>.json en S3 y el evento invoca la Lambda.

3 DECISIONES

D1 CAPACIDAD: NO DIMENSIONAR PARA EL PICO — R2 · R5

RESTRICCIÓN: El pico de la campaña dura pocos días, es incierto y casi todo es contenido estático; el presupuesto no paga capacidad ociosa. → **DECISIÓN:** Absorbo el pico en dos niveles en vez de comprar capacidad fija. → **MECANISMO:** CloudFront sirve la web desde caché, con S3 privado de origen; la API corre en un Auto Scaling Group de 2 a 6 EC2 con target tracking al 50% de CPU, y la víspera el mínimo sube a 4. → **CONSECUENCIA:** Lo estático no toca las EC2 y la API crece con la demanda; fuera de campaña pago 2 instancias. → **TRADE-OFF:** Ahorro capacidad ociosa (6 EC2 fijas costarían USD 61 más al mes), pero debo vigilar el escalamiento: por encima del techo de 6, la API se degrada.

Descarté ~~capacidad fija para el pico~~ y ~~serverless para toda la API~~. **Evidencia** Lab 06: la alarma de CPU tardó dos periodos de 5 min en dispararse; escalar por métrica toma minutos, por eso el mínimo sube la víspera.

D2 RESILIENCIA Y DATOS: NO PERDER PEDIDOS — R1 · R3 · R4

RESTRICCIÓN: Un pedido confirmado no se puede perder, la venta debe seguir si cae una instancia o una AZ, y el equipo no puede operar réplicas. → **DECISIÓN:** Todo en dos AZ y el estado fuera de las instancias, en RDS PostgreSQL Multi-AZ. → **MECANISMO:** El ALB envía solo a instancias sanas y el ASG reemplaza las que fallan el health check de tipo ELB; la sesión va en una cookie firmada. RDS copia cada escritura de forma síncrona al standby de la otra AZ y, si falla la primaria, mueve el endpoint. → **CONSECUENCIA:** Si cae una AZ, la venta sigue en 1 a 2 minutos sin perder pedidos (RPO 0); los backups restauran a cualquier punto de 7 días y se copian a us-east-2. → **TRADE-OFF:** Pago USD 26 más al mes que con Single-AZ y 2 instancias siempre encendidas; la base escala en vertical y puede ser el cuello de botella.

Descarté ~~PostgreSQL en EC2~~ y ~~DynamoDB~~. **Evidencia** Lab 02: el ALB sacó la instancia terminada en 35 s y el reemplazo atendía a los 2 min 40 s, sin cortar el servicio; con health check de tipo EC2 no habría visto una app colgada.

D3 RED Y SEGURIDAD: UNA SOLA PUERTA — R1 · R5

RESTRICCIÓN: Hay datos de clientes y pedidos, y todo lo expuesto a Internet es superficie de ataque. → **DECISIÓN:** CloudFront es la única entrada; la app y la base viven en subnets privadas, con salida mínima. → **MECANISMO:** Cada SG acepta solo a la capa anterior: CloudFront (prefix list) al ALB, el ALB a la app y la app a la base por 5432. Roles IAM mínimos, sin access keys, y token IAM para la base; KMS, ACM y CloudTrail. S3 por gateway endpoint y un solo NAT para parches. → **CONSECUENCIA:** Nadie llega directo al ALB ni a la base, y llegar no basta: IAM decide qué puede hacer cada rol. → **TRADE-OFF:** Más reglas que mantener. Un solo NAT ahorra USD 37 al mes; si cae su AZ, la otra sigue vendiendo, pero sin parches.

Descarté ~~un ALB abierto a Internet~~ y ~~un NAT por AZ~~. **Evidencia** Lab 04: A → B:8080 dio timeout al quitar la regla entre SG. Lab 05: con el mismo rol, GetObject pasó y PutObject dio AccessDenied.

D4 OPERACIÓN: TODO COMO CÓDIGO, ALARMAS QUE ACTÚAN — R4

RESTRICCIÓN: El equipo es pequeño: un cambio manual no se repite igual y una alarma que nadie mira no protege. → **DECISIÓN:** Toda la infraestructura en CloudFormation, y cada alarma con una acción o un responsable. → **MECANISMO:** Un template por capa (red, datos, app) crea, cambia con change set y borra, también el ambiente de la prueba de carga. El ASG escala solo; 5XX, instancias sanas, RDS, Lambda y presupuesto avisan por SNS al responsable de turno, con runbook. → **CONSECUENCIA:** Cada cambio es repetible y auditable, y cada señal termina en una acción. → **TRADE-OFF:** Cedo portabilidad frente a Terraform, y nada se toca en la consola: lo manual aparece como drift.

Descarté ~~Terraform~~ y ~~cambios en la consola~~. **Evidencia** Lab 06: CREATE_COMPLETE en 26 s, update sin reemplazo y delete sin residuos; la alarma llegó a ALARM, pero sin acciones solo informaba.

D5 MODERNIZACIÓN: SOLO LO INTERMITENTE, POR EVENTO — R2 · R4 · R6

RESTRICCIÓN: Los reportes son intermitentes y de bajo riesgo: no deben competir con la API en el pico ni justificar otro servidor. → **DECISIÓN:** Reportes por evento, S3 ObjectCreated → Lambda; la API, que es continua, sigue en EC2. → **MECANISMO:** La app escribe incoming/<id>.json y S3 invoca la Lambda de forma asíncrona (filtro incoming/ y .json); la Lambda escribe reports/<fecha>/<id>.json y un log con el mismo id. → **CONSECUENCIA:** Los reportes no usan las EC2 del pico ni un servidor dedicado, y el patrón sirve para el inventario. → **TRADE-OFF:** El evento llega al menos una vez y sin orden: escribir por id evita duplicados. Sumo una ruta de diagnóstico: evento, permiso y log.

Descarté ~~un proceso en las EC2~~ y ~~una VM dedicada~~. **Evidencia** Lab 07: incoming/order-001.json produjo una invocación y un log con el mismo key; .txt y archive/ no invocaron.

4 TRADE-OFFS, RIESGOS Y SUPUESTOS

	GANO	SACRIFICO	RIESGO QUE PERMANECE	LO DETECTO CON
D1	El pico estático no toca el cómputo	Capacidad sobre 6 EC2; frescura de la caché	Un pico mayor que el supuesto	p95 sobre 1 s con el ASG en 6
D2	Failover sin perder pedidos	USD 26 al mes; 2 EC2 siempre encendidas	La base como cuello de botella; una sola región	CPU y conexiones de RDS; ensayo en us-east-2
D3	Una sola puerta, sin llaves guardadas	Un NAT redundante (USD 37 al mes)	Una policy mal escrita; caída de la AZ del NAT	AccessDenied en los logs; AWS Health
D4	Cambios repetibles y auditables	Portabilidad del template	Un cambio manual fuera del template	Detección de drift
D5	Reportes sin servidor que operar	Inmediatez y orden de los eventos	Un evento que falla tras los reintentos	Alarma de errores de la Lambda

Resiliencia por escenario. Instancia: servicio continuo, RPO 0 · AZ: RTO 1 a 2 min, RPO 0 · Error humano: RTO 1 h, RPO 5 min · Región: RTO 4 h, RPO 1 h en us-east-2.

S1. El pico es unas 10 veces la carga normal por 3 a 5 días, sobre todo contenido estático. *Se valida con una prueba de carga y el cache hit ratio.*

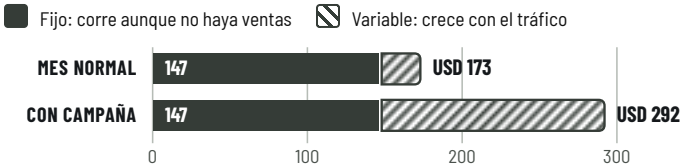
S2. La app es stateless: sesión en cookie firmada, carrito en la base y archivos en S3. *Se valida terminando una instancia sin perder carritos.*

S3. El pago es un checkout alojado con webhook: vender no necesita llamadas salientes. *Se valida con el proveedor; si no, un NAT por AZ (USD 37).*

S4. Los datos pueden alojarse en EE. UU. (Ley 1581 de 2012) y el equipo domina Linux y SQL. *Se valida con el área legal y con el equipo.*

S5. El negocio acepta 99,9% en campaña, p95 bajo 1 s, estos RTO y RPO, y unos USD 175 al mes (300 con campaña). *Se valida con el negocio; AWS Budgets avisa al 80%.*

5 COSTO Y EVOLUCIÓN



El 85% del mes normal es fijo (RDS, NAT, horas del ALB, IPv4 y 2 EC2): el piso lo ponen la base y la red, no el cómputo. En campaña manda el tráfico: CloudFront pasa de 16 a 88 y los créditos de CPU suman 22; las 4 EC2 extra cuestan 10. Palancas: la capa gratuita de CloudFront (1 TB y 10 millones de peticiones) deja USD 158 y 206; RDS reservado tras 3 meses de métricas estables; el ambiente de pruebas solo vive durante la prueba.

AWS Price List API en us-east-1, 21 y 23 de octubre de 2026: precios on-demand, CloudFront con tarifa de Sudamérica, 730 h al mes, sin capa gratuita. Normal: 2 EC2; 100 GB y 2 millones de peticiones en CloudFront; 30.000 pedidos. Campaña: 5 días con 6 EC2 al 50% de CPU; 600 GB y 9 millones de peticiones; 150.000 pedidos.

CUÁNDO	QUÉ	SEÑAL QUE LO DISPARA
Ahora	D1 a D5	Es lo que la campaña necesita
Tras la campaña	Inventario con el patrón de D5	Campaña cerrada y reportes estables
Siguiente	WAF y destino on-failure de la Lambda	Bots en los logs; errores de la Lambda
Siguiente	ElastiCache o réplica de lectura	CPU de RDS sobre 60% o p95 sobre 1 s
Después	ECS Fargate para la API	Despliegues semanales y equipo en contenedores
Después	Warm standby en us-east-2	RTO regional menor a 1 h y presupuesto para duplicar el fijo

No hago ahora contenedores, serverless para el núcleo, NoSQL ni multirregión activa: resuelven problemas que el caso no tiene y exigen operación que el equipo no tiene.